TypeScript高频面试题及解析

1. 什么是接口（interface），它的作用，接口的使用场景。接口和类型别名（Type Alias）的区别

接口是用于描述对象的形状的结构化类型。它定义了对象应该包含哪些属性和方法。在TypeScript中，接口可以用来约束对象的结构，以提高代码的可读性和维护性。例如：

```
interface Person {
    name: string;
    age: number;
}
function greet(person: Person) {
    return `Hello, ${person.name}!`;
}
```

`接口` 和 `类型别名` 的区别：

- `接口` 定义了一个契约，描述了对象的形状（属性和方法），以便在多个地方共享。它可以被类、对象和函数实现。
- `类型别名` 给一个类型起了一个新名字，便于在多处使用。它可以用于原始值、联合类型、交叉类型等。与接口不同，类型别名可以用于原始类型、联合类型、交叉类型等，而且还可以为任意类型指定名字。

2. 什么是泛型（generic），如何创建泛型函数和泛型类，实际用途

`泛型` 是一种在定义函数、类或接口时使用类型参数的方式，以增加代码的灵活性和重用性。在TypeScript中，可以使用来创建泛型。例如：

```
function identity<T>(arg: T): T {
    return arg;
}
// 调用泛型函数
let output = identity<string>("hello");
```

3. 枚举（enum）是什么，它的优势，应用案例。枚举和常量枚举的区别

枚举是一种对数字值集合进行命名的方式。它们可以增加代码的可读性，并提供一种便捷的方式来使用一组有意义的常量。例如：

```
enum Color {
    Red,
    Green,
    Blue
}

let selectedColor: Color = Color.Red;
```

枚举和常量枚举的区别:

- `枚举` 可以包含计算得出的值，而常量枚举则在编译阶段被删除，并且不能包含计算得出的值，它只能包含常量成员。
- `常量枚举` 在编译后会被删除，而普通枚举会生成真实的对象。

4. 如何处理可空类型（nullable types）和undefined类型，如何正确处理这些类型以避免潜在错误

在TypeScript中，可空类型是指一个变量可以存储特定类型的值，也可以存储 `null` 或 `undefined` 。（通过使用可空类型，开发者可以明确表达一个变量可能包含特定类型的值，也可能不包含值（即为 `null` 或 `undefined` ）。这有助于提高代码的可读性，并使得变量的可能取值范围更加清晰明了）。

为了声明一个可空类型，可以使用联合类型（Union Types），例如 `number | null` 或 `string | undefined` 。 例如：

```
let numberOrNull: number | null = 10;
numberOrNull = null; // 可以赋值为null

let stringOrUndefined: string | undefined = "Hello";
stringOrUndefined = undefined; // 可以赋值为undefined
```

5. 什么是联合类型和交叉类型

`联合类型` 表示一个值可以是多种类型中的一种，而 `交叉类型` 表示一个新类型，它包含了多个类型的特性。

- 联合类型示例：

```typescript
// typescript
let myVar: string | number;
myVar = "Hello"; // 合法
myVar = 123; // 合法
```

- 交叉类型示例：

```typescript
interface A {
  a(): void;
}
interface B {
  b(): void;
}
type C = A & B; // 表示同时具备 A 和 B 的特性
```

6. 什么是TypeScript中的声明文件（Declaration Files）

声明文件（通常以 `.d.ts` 扩展名结尾）用于描述已有 JavaScript 代码库的类型信息。它们提供了类型定义和元数据，以便在 TypeScript 项目中使用这些库时获得智能感知和类型安全。

7. 什么是命名空间（Namespace）和模块（Module）

`模块`

- 在一个大型项目中，可以将相关的代码组织到单独的文件，并使用模块来导入和导出这些文件中的功能。
- 在一个 Node.js 项目中，可以使用 import 和 export 关键字来创建模块，从而更好地组织代码并管理依赖关系。

`命名空间`

- 在面向对象的编程中，命名空间可以用于将具有相似功能或属性的类、接口等进行分组，以避免全局命名冲突。
- 这在大型的 JavaScript 或 TypeScript 应用程序中特别有用，可以确保代码结构清晰，并且不会意外地重复定义相同的名称。

`模块` 提供了一种组织代码的方式，使得我们可以轻松地在多个文件中共享代码，

`命名空间` 则提供了一种在全局范围内组织代码的方式，防止命名冲突。

模块示例:

```typescript
// greeter.ts
export function sayHello(name: string) {
```

```
  return `Hello, ${name}!`;
}
// app.ts
import { sayHello } from './greeter';
console.log(sayHello('John'));
```

命名空间示例:

```
// greeter.ts
namespace Greetings {
  export function sayHello(name: string) {
    return `Hello, ${name}!`;
  }
}
// app.ts
<reference path="greeter.ts" />
console.log(Greetings.sayHello('John'));
```

在上面的示例中:

- 使用模块时，我们可以使用 `export` 和 `import` 关键字来定义和引入模块中的函数或变量。
- 而在命名空间中，我们使用 namespace 来创建命名空间，并且需要在使用之前使用 `<reference path="file.ts" />` 来引入命名空间。

8. 什么是类型断言（Type Assertion）

类型断言允许程序员手动指定一个值的类型。这在需要明确告诉编译器某个值的类型时非常有用。

```
let someValue: any = "this is a string";
let strLength: number = (someValue as string).length;
```

9. TypeScript中的可选参数和默认参数是什么

- 可选参数允许函数中的某些参数不传值，在参数后面加上问号 `?` 表示可选。
- 默认参数允许在声明函数时为参数指定 `默认值` ，这样如果调用时未提供参数值，则会使用默认值。

可选参数示例:

```
function greet(name: string, greeting?: string) {
  if (greeting) {
```

```
    return `${greeting}, ${name}!`;
  } else {
    return `Hello, ${name}!`;
  }
}
```

默认参数示例：

```
function greet(name: string, greeting: string = "Hello") {
  return `${greeting}, ${name}!`;
}
```

10. 类型守卫（Type Guards）是什么

类型守卫是一种用于在运行时检查类型的技术，它允许开发人员在特定的作用域内缩小变量的范围，以确保正确推断类型。

```
function isString(test: any): test is string {
  return typeof test === "string";
}
if (isString(input)) {
  // input 在此代码块中被收窄为 string 类型
}
```

11. 索引类型（Index Types）是什么，好处有什么

索引类型允许我们在 TypeScript 中创建具有动态属性名称的对象，并且能够根据已知的键来获取相应的属性类型。 好处：

1.动态属性访问

> 在处理动态属性名的对象时，可以使用索引类型来实现类型安全的属性访问。例如，当从服务器返回的 JSON 数据中提取属性时，可以利用索引类型来确保属性名存在并获取其对应的类型。

2.代码重用

> 当需要创建通用函数来操作对象属性时，索引类型可以帮助我们实现更加通用和灵活的代码。例如，一个通用的函数可能需要根据传入的属性名称获取属性值，并进行特定的处理。

```
interface ServerData {
  id: number;
  name: string;
```

```
  age: number;
  // 可能还有其他动态属性
}
function getPropertyValue(obj: ServerData, key: keyof ServerData): void {
  console.log(obj\[key]); // 确保 obj\[key] 的类型是正确的 // 这里可以直接使用索引类型来获取属性值
}
```

3.动态扩展对象

> 当需要处理来自外部来源（比如 API 响应或数据库查询）的动态数据时，索引类型可以让我们轻松地处理这种情况，而不必为每个可能的属性手动定义类型。

```
interface DynamicObject {
  [key: string]: number | string; // 允许任意属性名，但属性值必须为 number 或 string 类型

}
function processDynamicData(data: DynamicObject): void {
  for (let key in data) {
    console.log(key + ": " + data\[key]); // 对任意属性进行处理
  }
}
```

4.类型安全性

> 索引类型可以增强代码的类型安全性，因为它们可以捕获可能的属性名拼写错误或键不存在的情况。

5.映射类型

> TypeScript 还提供了映射类型（Mapped Types）的概念，它们利用索引类型可以根据现有类型自动生成新类型。这在创建新类型时非常有用，特别是当需要在现有类型的基础上添加或修改属性时。

12. const和readonly的区别

当在TypeScript中使用 `const` 和 `readonly` 时，它们的行为有一些显著的区别：

- **const：**
  - `const` 用于声明常量值。一旦被赋值后，其值将不能被重新赋值或修改。
  - 常量必须在声明时就被赋值，并且该值不可改变。
  - 常量通常用于存储不会发生变化的值，例如数学常数或固定的配置值。

```
const PI = 3.14;
```

```
PI = 3.14159; // Error: 无法重新分配常量
```

- **readonly：**
  - `readonly` 关键字用于标记类的属性，表明该属性只能在类的构造函数或声明时被赋值，并且不能再次被修改。
  - `readonly` 属性可以在声明时或构造函数中被赋值，但之后不能再被修改。
  - `readonly` 属性通常用于表示对象的某些属性是只读的，防止外部代码修改这些属性的值。

```
class Person {
    readonly name: string;

    constructor(name: string) {
        this.name = name; // 可以在构造函数中赋值
    }
}

let person = new Person("Alice");
person.name = "Bob"; // Error: 无法分配到"name"，因为它是只读属性
```

总结来说， `const` 主要用于声明常量值，而 `readonly` 则用于标记类的属性使其只读。

13. TypeScript 中 any 类型的作用是什么，滥用会有什么后果

在TypeScript中， `any` 类型的作用是允许我们在编写代码时不指定具体的类型，从而可以接受任何类型的值。使用 `any` 类型相当于放弃了对该值的静态类型检查，使得代码在编译阶段不会对这些值进行类型检查。

主要情况下， `any` 类型的使用包括以下几点：

- 当我们不确定一个变量或表达式的具体类型时，可以使用any类型来暂时绕过类型检查。
- 在需要与动态类型的JavaScript代码交互时，可以使用any类型来处理这些动态类型的值。
- 有时候某些操作难以明确地定义其类型，或者需要较复杂的类型推导时，也可以使用any类型。

滥用的后果：

尽管any类型提供了灵活性，但由于它会放弃TypeScript的静态类型检查，因此滥用any类型可能会降低代码的健壮性和可维护性。当滥用 `any` 类型时，可能会导致以下后果：

1.代码可读性下降：

```
let data: any;
// 代码中的使用方式
```

```
data.someUnknownMethod(); // 在编译阶段不会报错，但实际上可能是一个错误
```

2.潜在的运行时错误：

```
let myVariable: any = 123;
myVariable.toUpperCase(); // 在编译阶段不会报错，但在运行时会引发错误
```

3.类型安全受损：

```
function add(x: any, y: any): any {
    return x + y; // 编译器无法推断返回值的具体类型
}
```

滥用 `any` 类型会导致代码失去了TypeScript强大的类型检查功能，带来了如下问题：

- 可能引入未知的运行时行为和错误。
- 降低了代码的可维护性和可读性，因为难以理解某些变量或参数的具体类型。

因此，在实际开发中，应尽量避免过度使用 `any` 类型。可以通过合适的类型声明、接口定义和联合类型等方式，提高代码的健壮性和可维护性。

14. TypeScript中的this有什么需要注意的

在TypeScript中，与JavaScript相比， `this` 的行为基本上是一致的。然而，TypeScript提供了类型注解和类型检查，可以帮助开发者更容易地理解和处理 `this` 关键字的使用。

> 在noImplicitThis为true 的情况下，必须声明 this 的类型，才能在函数或者对象中使用this。
>
> Typescript中箭头函数的 this 和 ES6 中箭头函数中的 this 是一致的。

在TypeScript中，当将 `noImplicitThis` 设置为 `true` 时，意味着在函数或对象中使用this时，必须显式声明 `this` 的类型。这一设置可以帮助开发者更明确地指定this的类型，以避免因为隐式的 `this` 引用而导致的潜在问题。

具体来说，如果将 `noImplicitThis` 设置为 `true` ，则在下列情况下必须显式声明this的类型：

- 在函数内部使用this时，需要使用箭头函数或显示绑定this。
- 在某些类方法或对象方法中，需要明确定义this的类型。

示例代码如下所示：

```
class MyClass {
  private value: number = 42;
```

```
3
4     public myMethod(this: MyClass) {
5       console.log(this.value);
6     }
7
8     public myMethod2 = () => {
9       console.log(this.value);
10    }
11  }
12
13  let obj = new MyClass();
14  obj.myMethod(); // 此处必须传入合适的 this 类型
```

通过将 `noImplicitThis` 设置为 `true` ，TypeScript要求我们在使用 `this` 时明确指定其类型，从而在编译阶段进行更严格的类型检查，帮助避免一些可能出现的错误和不确定性。

注： `noImplicitThis` 是TypeScript编译器的一个配置选项，用于控制在函数或对象方法中使用 `this` 时的严格性。当将 `noImplicitThis` 设置为 `true` 时，意味着必须显式声明 `this` 的类型，否则会触发编译错误。

15. TypeScript数据类型

在TypeScript中，常见的数据类型包括以下几种：

- **基本类型**：
  - `number` : 表示数字，包括整数和浮点数。
  - `string` : 表示文本字符串。
  - `boolean` : 表示布尔值，即 `true` 或 `false` 。
  - `null` 、 `undefined` : 分别表示null和undefined。
  - `symbol` : 表示唯一的、不可变的值。
- **复合类型**：
  - `array` : 表示数组，可以使用 `number[]` 或 `Array<number>` 来声明其中元素的类型。
  - `tuple` : 表示元组，用于表示固定数量和类型的数组。
  - `enum` : 表示枚举类型，用于定义具名常量集合。
- **对象类型**：
  - `object` : 表示非原始类型，即除number、string、boolean、symbol、null或undefined之外的类型。
  - `interface` : 用于描述对象的结构，并且可以重复使用。
- **函数类型**：

- `function`：表示函数类型。
- `void`：表示函数没有返回值。
- `any`：表示任意类型。

- **高级类型：**
  - `union types`：表示一个值可以是几种类型之一。
  - `intersection types`：表示一个值同时拥有多种类型的特性。

16. interface可以给Function/Array/Class（Indexable)做声明吗

在TypeScript中，`interface` 可以用来声明函数、数组和类（具有索引签名的类）。下面是一些示例代码：

1. **Interface 声明函数**

```
interface MyFunc {
  (x: number, y: number): number;
}

let myAdd: MyFunc = function(x, y) {
  return x + y;
};
```

在上述示例中，`MyFunc` 接口描述了一个函数类型，该函数接受两个参数并返回一个数字。

2. **Interface 声明数组**

```
interface StringArray {
  [index: number]: string;
}

let myArray: StringArray;
myArray = ["Bob", "Alice"];
```

上面的示例中，`StringArray` 接口描述了一个具有数字索引签名的字符串数组。意味着我们可以通过数字索引来访问数组元素。

3. **Interface 声明类（Indexable）**

```
interface StringDictionary {
  [index: string]: string;
```

```
}

let myDict: StringDictionary = {
  "name": "John",
  "age": "30"
};
```

在这个例子中，`StringDictionary`接口用于描述具有字符串索引签名的类或对象。这使得我们可以像操作字典一样使用对象的属性。

综上：TypeScript中的`interface`可以被用来声明函数、数组和具有索引签名的类，从而帮助我们定义和限定这些数据结构的形式和行为。

17. TypeScript中的协变、逆变、双变和抗变是什么

在TypeScript中，`协变（Covariance）`、`逆变（Contravariance）`、`双变（Bivariance）`和`抗变（Invariance`是与类型相关的概念，涉及到参数类型的子类型关系。下面对这些概念进行解释，并提供示例代码。

协变（Covariance）

- **区别**：协变意味着子类型可以赋值给父类型。
- **应用场景**：数组类型是协变的，因此可以将子类型的数组赋值给父类型的数组。

`协变`表示类型T的子类型可以赋值给类型U，当且仅当T是U的子类型。在TypeScript中，`数组`是协变的，这意味着可以将子类型的数组赋值给父类型的数组。

```
let subtypes: string[] = ["hello", "world"];
let supertype: Object[] = subtypes; // 数组是协变的，这是合法的
```

逆变（Contravariance）

- **区别**：逆变意味着超类型可以赋值给子类型。
- **应用场景**：函数参数类型是逆变的，因此可以将超类型的函数赋值给子类型的函数。

`逆变`表示类型T的超类型可以赋值给类型U，当且仅当T是U的子类型。在TypeScript中，`函数参数`是逆变的，这意味着可以将超类型的函数赋值给子类型的函数。

```
type Logger<T> = (arg: T) => void;
let logNumber: Logger<number> = (x: number) => console.log(x);
let logAny: Logger<any> = logNumber; // 函数参数是逆变的，这是合法的
```

双变（Bivariance）

- **区别**：双变允许参数类型既是协变又是逆变的。
- **应用场景**：对象类型是双变的，这意味着可以将子类型的对象赋值给父类型的对象，同时也可以将超类型的对象赋值给子类型的对象。

`双变` 允许参数类型既是 `协变` 又是 `逆变` 的。在TypeScript中， `普通对象类型` 是双变的，这意味着可以将子类型的对象赋值给父类型的对象，并且可以将超类型的对象赋值给子类型的对象。

```
interface Animal {
  name: string;
}

interface Dog extends Animal {
  breed: string;
}

let animal: Animal = { name: "Animal" };
let dog: Dog = { name: "Dog", breed: "Labrador" };

animal = dog; // 对象类型是双变的，这是合法的
dog = animal; // 对象类型是双变的，这也是合法的
```

抗变（Invariance）

- **区别**：抗变表示不允许类型之间的任何赋值关系。
- **应用场景**：通常情况下，基本类型和类类型是抗变的。

`抗变` 表示不允许类型T和U之间的任何赋值关系，即T既不是U的子类型，也不是U的超类型。在TypeScript中，一般情况下， `基本类型` 和 `类类型` 是抗变的。

```
let x: string = "hello";
let y: string = x; // 这是合法的

let a: Animal = { name: "Animal" };
let b: Animal = a; // 这也是合法的
```

18. TypeScript中的静态类型和动态类型有什么区别

- `静态类型` 是在 **编译期间** 进行类型检查，可以在编辑器或 IDE 中发现大部分类型错误。
- `动态类型` 是在 **运行时** 才确定变量的类型，通常与动态语言相关联。

静态类型（Static Typing）

- **定义**：静态类型是指在编译阶段进行类型检查的类型系统，通过类型注解或推断来确定变量、参数和返回值的类型。
- **特点**：静态类型能够在编码阶段就发现大部分类型错误，提供了更好的代码健壮性和可维护性。
- **优势**：可以在编辑器或 IDE 中实现代码提示、自动补全和类型检查，帮助开发者减少错误并提高代码质量。

动态类型（Dynamic Typing）

- **定义**：动态类型是指在运行时才确定变量的类型，通常与动态语言相关联，允许同一个变量在不同时间引用不同类型的值。
- **特点**：动态类型使得变量的类型灵活多变，在运行时可以根据上下文或条件动态地改变变量的类型。
- **优势**：动态类型可以带来更大的灵活性，适用于一些需要频繁变化类型的场景。

区别总结

- **时机差异**：静态类型在编译期间进行类型检查，而动态类型是在运行时才确定变量的类型。
- **代码稳定性**：静态类型有助于在编码阶段发现大部分类型错误，提高代码稳定性；动态类型对类型的要求较为灵活，但可能增加了代码的不确定性。
- **使用场景**：静态类型适合于大型项目和团队，能够提供更强的类型安全性；动态类型适用于快速原型开发和灵活多变的场景，能够更快地迭代和测试代码。

4. 介绍TypeScript中的可选属性、只读属性和类型断言

- **可选属性** 使用 `?` 来标记一个属性可以存在，也可以不存在。
- **只读属性** 使用 `readonly` 关键字来标记一个属性是只读的。
- **类型断言** 允许将一个实体强制指定为特定的类型，使用 `<Type>` 或 `value as Type`。

代码示例：

```
// 可选属性
interface Person {
  name: string;
  age?: number; // 可选属性
}

// 只读属性
interface Point {
  readonly x: number;
  readonly y: number;
}
let p1: Point = { x: 10, y: 20 };
p1.x = 5; // Error: 只读属性无法重新赋值

```

```
// 类型断言
let someValue: any = "hello";
let strLength: number = (someValue as string).length;
```

19. TypeScript 中的模块化是如何工作的，举例说明

答案：

- TypeScript 中使用 ES6 模块系统，可以使用 `import` 和 `export` 关键字来导入和导出模块。
- 可以通过 `export default` 导出默认模块，在导入时可以使用 `import moduleName from 'modulePath'`。

代码示例：

```
// math.ts
export function sum(a: number, b: number): number {
  return a + b;
}
export function subtract(a: number, b: number): number {
  return a - b;
}

// app.ts
import { sum, subtract } from './math';
console.log(sum(3, 5)); // 输出 8
```

20. 如何约束泛型参数的类型范围

可以使用泛型约束（`extends关键字`）来限制泛型参数的类型范围，确保泛型参数符合某种特定的条件。

代码示例：

```
interface Lengthwise {
  length: number;
}
function loggingIdentity<T extends Lengthwise>(arg: T): T {
  console.log(arg.length);
  return arg;
}
loggingIdentity({length: 10, value: 3});  // 参数满足 Lengthwise 接口要求，可以正常调用
```

21. 什么是泛型约束中的 keyof 关键字，举例说明其用法。

- `keyof` 是 TypeScript 中用来获取对象类型所有键（属性名）的操作符。
- 可以使用 `keyof` 来定义泛型约束，限制泛型参数为某个对象的键。

代码示例：

```
function getProperty<T, K extends keyof T>(obj: T, key: K) {
  return obj[key];
}
let x = { a: 1, b: 2, c: 3 };
getProperty(x, "a"); // 正确
getProperty(x, "d"); // 错误: Argument of type '"d"' is not assignable to parameter of type '"a" | "b" | "c"'
```

22. 什么是条件类型（conditional types），能够举例说明其使用场景吗

- 条件类型是 TypeScript 中的高级类型操作符，可以根据一个类型关系判断结果类型。
- 例如，可以使用条件类型实现一个类型过滤器，根据输入类型输出不同的结果类型。

代码示例：

```
type NonNullable<T> = T extends null | undefined ? never : T;
type T0 = NonNullable<string | null>;  // string
type T1 = NonNullable<string | null | undefined>;  // string
type T2 = NonNullable<string | number | null>;  // string | number
```

23. 什么是装饰器，有什么作用，如何在TypeScript中使用类装饰器

- `装饰器` 是一种特殊类型的声明，可以附加到类、方法、访问符、属性或参数上，以修改其行为。
- 在 TypeScript 中，装饰器提供了一种在声明时定义如何处理类的方法、属性或参数的机制。

如何在 TypeScript 中使用类装饰器：

```
function classDecorator<T extends { new(...args: any[]): {} }>(constructor: T) {
  return class extends constructor {
    newProperty = "new property";
    hello = "override";
  };
}

```

```
@classDecorator
class Greeter {
  property = "property";
  hello: string;
  constructor(m: string) {
    this.hello = m;
  }
}
console.log(new Greeter("world")); // 输出 { property: 'property', hello: 'override', newProperty: 'new property' }
```

24. 类装饰器和方法装饰器的执行顺序是怎样的

- 当有 `多个装饰器应用于同一个声明` 时（比如一个类中的方法），它们将按照 `自下而上` 的顺序应用。
- 对于 `方法装饰器` ，从顶层方法开始 `依次向下` `递归调用` 方法装饰器函数。

25. 装饰器工厂是什么，请给出一个装饰器工厂的使用示例

- `装饰器工厂` 是一个返回装饰器的函数。它可以接受参数，并根据参数动态生成装饰器。

以下是一个简单的装饰器工厂示例：

```
function color(value: string) {
  return function (target: any, propertyKey: string) {
    // ... 在此处使用 value 和其他参数来操作装饰目标
  };
}

class Car {
  @color('red')
  brand: string;
}
```

26. 说说对 TypeScript 中命名空间与模块的理解？区别？

一、模块

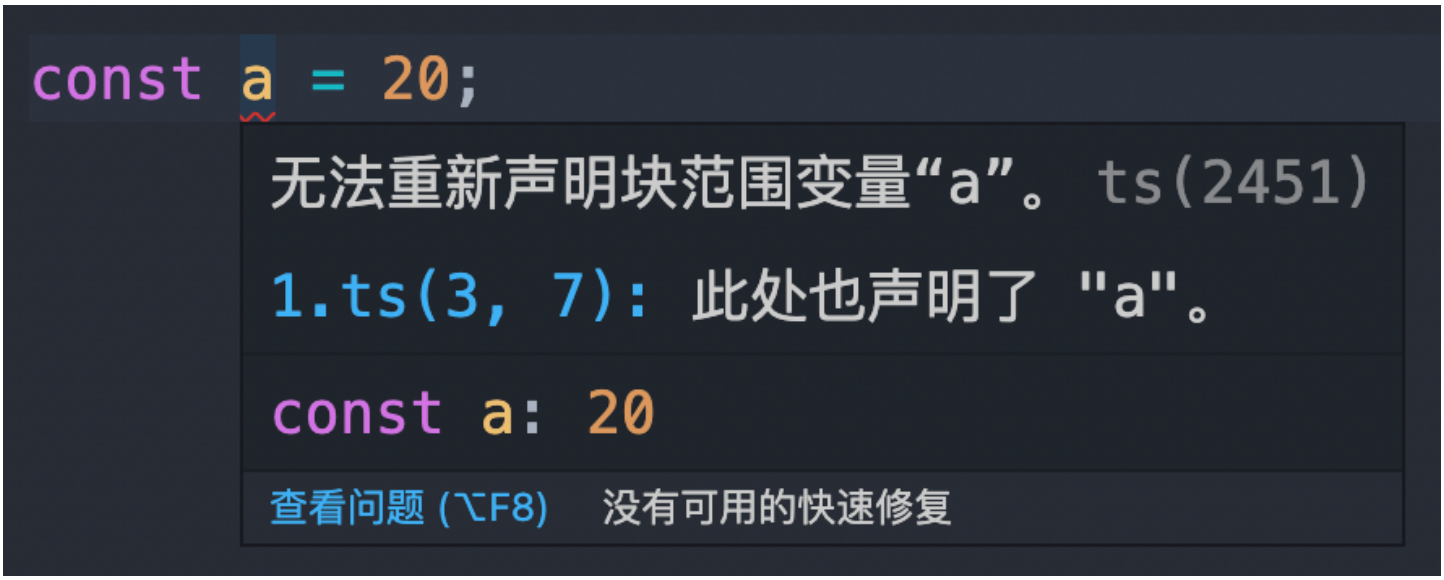
`TypeScript` 与 `ECMAScript` 2015 一样，任何包含顶级 `import` 或者 `export` 的文件都被当成一个模块

相反地，如果一个文件不带有顶级的 `import` 或者 `export` 声明，那么它的内容被视为全局可见的

例如我们在在一个 `TypeScript` 工程下建立一个文件 `1.ts` ，声明一个变量 `a` ，如下：

```
const a = 1
```

然后在另一个文件同样声明一个变量 `a` ，这时候会出现错误信息



提示重复声明 `a` 变量，但是所处的空间是全局的

如果需要解决这个问题，则通过 `import` 或者 `export` 引入模块系统即可，如下：

```
const a = 10;
export default a
```

在 `typescript` 中， `export` 关键字可以导出变量或者类型，用法与 `es6` 模块一致，如下：

```
export const a = 1
export type Person = {
    name: String
}
```

通过 `import` 引入模块，如下：

```
import { a, Person } from './export';
```

二、命名空间

命名空间一个最明确的目的就是解决重名问题

命名空间定义了标识符的可见范围，一个标识符可在多个名字空间中定义，它在不同名字空间中的含义是互不相干的

这样，在一个新的名字空间中可定义任何标识符，它们不会与任何已有的标识符发生冲突，因为已有的定义都处于其他名字空间中

`TypeScript` 中命名空间使用 `namespace` 来定义，语法格式如下：

```
namespace SomeNameSpaceName {
   export interface ISomeInterfaceName {      }
   export class SomeClassName {      }
}
```

以上定义了一个命名空间 `SomeNameSpaceName` ，如果我们需要在外部可以调用 `SomeNameSpaceName` 中的类和接口，则需要在类和接口添加 `export` 关键字

使用方式如下：

```
SomeNameSpaceName.SomeClassName
```

命名空间本质上是一个对象，作用是将一系列相关的全局变量组织到一个对象的属性，如下：

```
namespace Letter {
  export let a = 1;
  export let b = 2;
  export let c = 3;
  // ...
  export let z = 26;
}
```

编译成 `js` 如下：

```
var Letter;
(function (Letter) {
    Letter.a = 1;
    Letter.b = 2;
    Letter.c = 3;
    // ...
    Letter.z = 26;
```

```
})(Letter || (Letter = {}));
```

三、区别

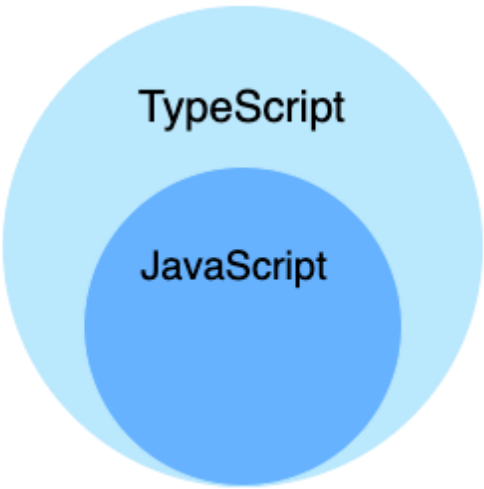
- 命名空间是位于全局命名空间下的一个普通的带有名字的 JavaScript 对象，使用起来十分容易。但就像其它的全局命名空间污染一样，它很难去识别组件之间的依赖关系，尤其是在大型的应用中
- 像命名空间一样，模块可以包含代码和声明。 不同的是模块可以声明它的依赖
- 在正常的TS项目开发过程中并不建议用命名空间，但通常在通过 d.ts 文件标记 js 库类型的时候使用命名空间，主要作用是给编译器编写代码的时候参考使用

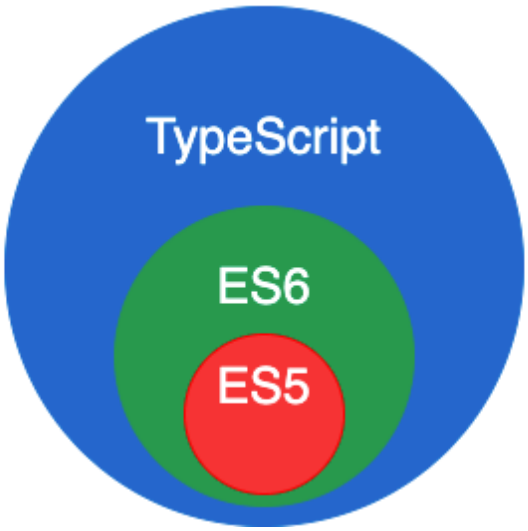
27. 说说你对 typescript 的理解？与 javascript 的区别？

一、是什么

`TypeScript` 是 `JavaScript` 的类型的超集，支持 `ES6` 语法，支持面向对象编程的概念，如类、接口、继承、泛型等

> 超集，不得不说另外一个概念，子集，怎么理解这两个呢，举个例子，如果一个集合A里面的的所有元素集合B里面都存在，那么我们可以理解集合B是集合A的超集，集合A为集合B的子集





其是一种静态类型检查的语言，提供了类型注解，在代码编译阶段就可以检查出数据类型的错误

同时扩展了 `JavaScript` 的语法，所以任何现有的 `JavaScript` 程序可以不加改变的在 `TypeScript` 下工作

为了保证兼容性，`typescript` 在编译阶段需要编译器编译成纯 `Javascript` 来运行，是为大型应用之开发而设计的语言，如下：

`tsx` 文件如下：

```
const hello : string = "Hello World!"
```

```
console.log(hello)
```

编译文件后：

```
const hello = "Hello World!"
console.log(hello)
```

二、特性

`typescript` 的特性主要有如下：

- 类型批注和编译时类型检查 ： 在编译时批注变量类型
- 类型推断：ts中没有批注变量类型会自动推断变量的类型
- 类型擦除：在编译过程中批注的内容和接口会在运行时利用工具擦除
- 接口：ts中用接口来定义对象类型
- 枚举：用于取值被限定在一定范围内的场景
- Mixin：可以接受任意类型的值
- 泛型编程：写代码时使用一些以后才指定的类型
- 名字空间：名字只在该区域内有效，其他区域可重复使用该名字而不冲突
- 元组：元组合并了不同类型的对象，相当于一个可以装不同类型数据的数组
- ...

类型批注

通过类型批注提供在编译时启动类型检查的静态类型，这是可选的，而且可以忽略而使用 `JavaScript` 常规的动态类型

```
function Add(left: number, right: number): number {
 return left + right;
}
```

对于基本类型的批注是 `number` 、 `bool` 和 `string` ，而弱或动态类型的结构则是 `any` 类型

类型推断

当类型没有给出时，TypeScript编译器利用类型推断来推断类型，如下：

```
let str = 'string'
```

变量 `str` 被推断为字符串类型，这种推断发生在初始化变量和成员，设置默认参数值和决定函数返回值时

如果由于缺乏声明而不能推断出类型，那么它的类型被视作默认的动态 `any` 类型

接口

接口简单来说就是用来描述对象的类型 数据的类型有 `number` 、 `null` 、 `string` 等数据格式，对象的类型就是用接口来描述的

```
interface Person {
    name: string;
    age: number;
}
let tom: Person = {
    name: 'Tom',
    age: 25
};
```

三、区别

- TypeScript 是 JavaScript 的超集，扩展了 JavaScript 的语法
- TypeScript 可处理已有的 JavaScript 代码，并只对其中的 TypeScript 代码进行编译
- TypeScript 文件的后缀名 .ts （.ts，.tsx，.dts），JavaScript 文件是 .js
- 在编写 TypeScript 的文件的时候就会自动编译成 js 文件

更多的区别如下图所示：

	JavaScript	TypeScript
语言	脚本语言	面向对象编程语言
学习难度	灵活易学	需要有脚本编程经验
类型	轻量级解释编程语言	强类型的面向对象编程语言
客户端/服务	客户端服务端都有	侧重客户端
拓展名	.js	.ts 或 .tsx
耗时	更快	编译代码需要些时间
数据绑定	没有类型和接口的概念	使用类型和接口表示数据
语法	所有的语句都写在脚本标签内。浏览器将脚本标签内的文本识别为脚本	一个 TypeScript 程序由模块、方法、变量、语句、表达式和注释构成
静态类型	JS 中没有静态类型的概念	支持静态类型
模块支持	不支持模块	支持模块
接口	没有接口	支持接口
可选参数方法	不支持	支持
原型	没有这种特性	支持原型特性

28. Typescript中泛型是什么?

一、是什么

泛型程序设计（generic programming）是程序设计语言的一种风格或范式

泛型允许我们在强类型程序设计语言中编写代码时使用一些以后才指定的类型，在实例化时作为参数指明这些类型 在 `typescript` 中，定义函数，接口或者类的时候，不预先定义好具体的类型，而在使用的时候在指定类型的一种特性

假设我们用一个函数，它可接受一个 `number` 参数并返回一个 `number` 参数，如下写法：

```
function returnItem (para: number): number {
    return para
}
```

如果我们打算接受一个 `string` 类型，然后再返回 `string` 类型，则如下写法：

```
function returnItem (para: string): string {
    return para
```

```
}
```

上述两种编写方式，存在一个最明显的问题在于，代码重复度比较高

虽然可以使用 `any` 类型去替代，但这也并不是很好的方案，因为我们的目的是接收什么类型的参数返回什么类型的参数，即在运行时传入参数我们才能确定类型

这种情况就可以使用泛型，如下所示：

```
function returnItem<T>(para: T): T {
    return para
}
```

可以看到，泛型给予开发者创造灵活、可重用代码的能力

二、使用方式

泛型通过 `<>` 的形式进行表述，可以声明：

- 函数
- 接口
- 类

函数声明

声明函数的形式如下：

```
function returnItem<T>(para: T): T {
    return para
}
```

定义泛型的时候，可以一次定义多个类型参数，比如我们可以同时定义泛型 `T` 和 泛型 `U`：

```
function swap<T, U>(tuple: [T, U]): [U, T] {
    return [tuple[1], tuple[0]];
}
swap([7, 'seven']); // ['seven', 7]
```

接口声明

声明接口的形式如下：

```
interface ReturnItemFn<T> {
    (para: T): T
}
```

那么当我们想传入一个number作为参数的时候，就可以这样声明函数:

```
const returnItem: ReturnItemFn<number> = para => para
```

类声明

使用泛型声明类的时候，既可以作用于类本身，也可以作用与类的成员函数

下面简单实现一个元素同类型的栈结构，如下所示：

```
class Stack<T> {
    private arr: T[] = []
    public push(item: T) {
        this.arr.push(item)
    }
    public pop() {
        this.arr.pop()
    }
}
```

使用方式如下：

```
const stack = new Stacn<number>()
```

如果上述只能传递 `string` 和 `number` 类型，这时候就可以使用 `<T extends xx>` 的方式猜实现约束泛型，如下所示：

```
type Params = string | number

class Stack<T extends Params> {
    private arr: T[] = []

    public push(item: T) {
        this.arr.push(item)
    }

    public pop() {
        this.arr.pop()
    }
}

const stack = new Stack<boolean>()
```

类型“boolean”不满足约束“Params”。 ts(2344)

查看问题 (⌥F8)　没有可用的快速修复

除了上述的形式，泛型更高级的使用如下：

例如要设计一个函数，这个函数接受两个参数，一个参数为对象，另一个参数为对象上的属性，我们通过这两个参数返回这个属性的值

这时候就设计到泛型的索引类型和约束类型共同实现

索引类型、约束类型

索引类型 `keyof T` 把传入的对象的属性类型取出生成一个联合类型，这里的泛型 U 被约束在这个联合类型中，如下所示：

```
function getValue<T extends object, U extends keyof T>(obj: T, key: U) {
  return obj[key] // ok
}
```

上述为什么需要使用泛型约束，而不是直接定义第一个参数为 `object` 类型，是因为默认情况 `object` 指的是 `{}` ，而我们接收的对象是各种各样的，一个泛型来表示传入的对象类型，比如 `T extends object`

使用如下图所示：

```
function getValue<T extends object, U extends keyof T>(obj: T, key: U) {
    return obj[key] // ok
  }

const a = {
    name: 'huihui',
    age: 18
}
```

```
getValue(obj: { name: string; age: number; }, key: "name" | "age"): string | number
```

```
getValue(a,)
```

多类型约束

例如如下需要实现两个接口的类型约束：

```
interface FirstInterface {
  doSomething(): number
}
interface SecondInterface {
  doSomethingElse(): string
}

```

可以创建一个接口继承上述两个接口，如下：

```
interface ChildInterface extends FirstInterface, SecondInterface {}
```

正确使用如下：

```
class Demo<T extends ChildInterface> {
  private genericProperty: T
  constructor(genericProperty: T) {
    this.genericProperty = genericProperty
  }
  useT() {
    this.genericProperty.doSomething()
    this.genericProperty.doSomethingElse()
  }
```

```
}
```

通过泛型约束就可以达到多类型约束的目的

三、应用场景

通过上面初步的了解，后述在编写 `typescript` 的时候，定义函数，接口或者类的时候，不预先定义好具体的类型，而在使用的时候在指定类型的一种特性的时候，这种情况下就可以使用泛型

灵活的使用泛型定义类型，是掌握 `typescript` 必经之路

29. TypeScript中有哪些声明变量的方式?

1. 使用 `let` 关键字:

```
let x: number = 10;
```

1. 使用 `const` 关键字来声明常量:

```
const pi: number = 3.14;
```

1. 使用 `var` 关键字（不推荐，通常在ES6之前使用）:

```
var name: string = "John";
```

1. 使用函数作用域声明变量:

```
function example() {
    var localVar: number = 42;
}
```

1. 使用函数参数的方式声明变量:

```
function greet(name: string) {
    console.log(`Hello, ${name}!`);
}
```

1. 使用对象字面量声明变量：

```
let person: { name: string, age: number } = { name: "Alice", age: 30 };
```

1. 使用数组字面量声明数组变量：

```
let numbers: number[] = [1, 2, 3, 4, 5];
```

1. 使用接口声明对象的结构：

```
interface Person {
    name: string;
    age: number;
}
let person: Person = { name: "Bob", age: 25 };
```

1. 使用类来声明对象：

```
class Animal {
    constructor(public name: string, public species: string) {}
}
let cat: Animal = new Animal("Fluffy", "Cat");
```

30. 什么是Typescript的方法重载?

在TypeScript中，方法重载（Method Overloading）是一种允许函数在不同参数数量或参数类型下具有不同的返回类型或行为的特性。这允许您以一种更灵活的方式定义函数，并根据传入的参数类型或数量来选择适当的行为或返回类型。

方法重载通常用于提供更加严格的类型检查和更好的类型推断，以及在代码中提供更清晰的接口。它使得函数可以根据不同的参数签名，提供不同的实现方式，而无需使用额外的运行时检查。

要定义方法重载，您需要按照以下步骤进行：

1. 首先，定义一个函数的多个签名（overload signatures）。每个签名包含一个参数列表和一个返回类型。
2. 然后，定义一个实际的函数体，这个函数体实现了多个签名所涵盖的不同情况。

这里有一个简单的例子，演示了如何在TypeScript中使用方法重载：

```
function greet(name: string): string;
function greet(age: number): string;
function greet(value: string | number): string {
  if (typeof value === "string") {
    return `Hello, ${value}!`;
  } else {
    return `You are ${value} years old!`;
  }
}
console.log(greet("Lydia")); // Output: "Hello, Lydia!"
console.log(greet(30)); // Output: "You are 30 years old!"
```

上面定义了 `greet` 函数的两个不同的签名：一个接受 `string` 类型参数，另一个接受 `number` 类型参数。然后，我们实现了一个函数体，根据传入的参数类型进行相应的处理。

使用方法重载，TypeScript能够更好地检查函数调用，以确保传递的参数类型与预期的类型相符，并提供适当的类型推断，从而增加代码的类型安全性和可读性。

31. 请实现 sleep 方法

```
/**
 * @file 返回一个 Promise，并在 ms 毫秒后 Promise 变为完成状态
 */

export function sleep(ms: number): Promise<undefined> {
  // 参考答案
  return new Promise(
    (
      resolve: (value: undefined) => void,
      reject: (value: undefined) => void
    ) => {
      setTimeout(() => {
        resolve(undefined);
      }, ms);
    }
  );
}

async function main() {
  console.log("a");
  await sleep(1000);
  console.log("b");
  await sleep(1000);
  console.log("c");
```

```
}
main();

export default {};
```

32. typescript 中的 is 关键字有什么用?

TypeScript 中的 `is` 关键字用于类型保护，可以在运行时判断一个对象是否属于某个类型，并根据不同的类型执行不同的逻辑。

具体来说，`is` 关键字通常和 `instanceof` 运算符一起使用，用于判断一个对象是否是某个类的实例。例如：

```
class Animal {
    name: string;
    constructor(name: string) {
        this.name = name;
    }
}
class Dog extends Animal {
    breed: string;
    constructor(name: string, breed: string) {
        super(name);
        this.breed = breed;
    }
}
function isDog(animal: Animal): animal is Dog {
    return (animal as Dog).breed !== undefined;
}
let a1 = new Animal("Tom");
let d1 = new Dog("Tony", "Poodle");
console.log(isDog(a1));  // false
console.log(isDog(d1));  // true
```

在上面的代码中，我们定义了一个 `isDog` 函数，它接受一个 `Animal` 类型的参数，返回值是一个布尔值。如果这个参数是 `Dog` 类型的实例，则返回 `true`；否则返回 `false`。注意，这里我们使用 `animal is Dog` 语法来显式地指定返回值类型为布尔值，表示这个函数就是一个类型谓词函数。

在 `isDog` 函数中，我们通过判断传入的 `animal` 参数是否含有 `breed` 属性，来判断它是否是 `Dog` 类型的实例。如果是，则返回 `true`；否则返回 `false`。

最后，我们可以通过调用 `isDog` 函数来判断一个对象是否是 `Dog` 类型的实例，并根据不同的类型执行相应的逻辑。

33. TypeScript支持的访问修饰符有哪些?

TypeScript支持访问修饰符 public，private 和 protected，它们决定了类成员的可访问性。

- 公共（public），类的所有成员，其子类以及该类的实例都可以访问。
- 受保护（protected），该类及其子类的所有成员都可以访问它们。 但是该类的实例无法访问。
- 私有（private），只有类的成员可以访问它们。

如果未指定访问修饰符，则它是隐式公共的，因为它符合 JavaScript 的便利性。

34. 请实现 myMap 方法

```
/**
 * @file 实现数组 map 方法
 */

function myMap<T, R>(arr: T[], callbackFn: (v: T) => R): R[] {
  // 参考答案
  var arr1: R[] = [];
  for (var i = 0; i < arr.length; i++) {
    arr1[i] = callbackFn(arr[i]);
  }
  return arr1;
}
// 测试
console.log(myMap([1, 2, 3], (v) => v * 2)); // [2, 4, 6]

export default {};
```

35. 请实现 treePath 方法

```
/**
 * @file 二叉树所有路径
 */

type Tree = {
  value: number;
  left?: Tree;
  right?: Tree;
};

const tree: Tree = {
  value: 1,
```

```
  left: {
    value: 2,
    right: { value: 5 },
  },
  right: { value: 3 },
};

function treePath(root: Tree): string[] {
  const answer: [] = [];
  let tmp: [][] = [];
  const travel = (r: Tree) => {
    if (r == null) {
      return;
    }
    //@ts-ignore
    tmp.push(r.value);
    if (r.left == null && r.right == null) {
      //@ts-ignore
      answer.push(tmp);
      tmp = [tmp[0]];
      return;
    }
    if (r.left) travel(r.left);
    if (r.right) travel(r.right);
  };
  travel(root);
  //@ts-ignore
  return answer.map((t) => t.join("->"));
}
console.log(treePath(tree)); // [ '1->2->5', '1->3' ]

export default {};
```

36. 请实现 product 方法

/**

- @file 计算数组笛卡尔积 */

// 示例 product([1, 2], [3, 4]) // [[1, 3], [1, 4], [2, 3], [2, 4]]

function product(xList: number[], yList: number[]): [number, number][] { // 补全内部实现 throw new Error('功能待实现'); }

export default {}

```
/**
```

```
 * @file 计算数组笛卡尔积
 */

// 示例
console.log(product([1, 2], [3, 4])); // [[1, 3], [1, 4], [2, 3], [2, 4]]

function product(xList: number[], yList: number[]): [number, number][] {
  // 参考答案
  return xList.reduce((v, t) => {
    return v.concat(yList.map((item) => [t, item]));
  }, [] as [number, number][]);
}

export default {};
```

37. 请实现 myAll 方法

```
/**
 * @file 实现 PromiseAll 方法
 */

import { sleep } from "./8.sleep";

async function myAll<T extends unknown[] | []>(
  values: T
): Promise<{ [P in keyof T]: Awaited<T[P]> }> {
  // 补全此处代码，使用 Promise.all 以外的语法完成
  // throw new Error('功能待实现');
  var arr = [];
  for (var i = 0; i < values.length; i++) {
    arr.push(await values[i]);
  }
  return arr as { [P in keyof T]: Awaited<T[P]> };
}

// 一秒钟后返回结果 value
async function request(value: string) {
  await sleep(1000);
  return value;
}
async function main() {
  console.log("start");
  const res = await myAll([request("a"), request("b"), request("c")]);
  console.log(res); // 预期输出 start 一秒后输出 ['a', 'b', 'c']
}
```

```
main();

export default {};
```

38. 请实现 sum 方法

```
/**
 * @file 假设加法是一个异步过程，如何计算多个数组之和?
 */
function sleep(ms: number) {
  return new Promise((r) => {
    setTimeout(() => {
      r(undefined);
    }, ms);
  });
}

async function asyncAdd(a: number, b: number) {
  await sleep(1000);
  return a + b;
}

async function sum(arr: number[]): Promise<number> {
  // 参考答案
  var s: number = arr[0];
  for (var i = 1; i < arr.length; i++) {
    s = await asyncAdd(s, arr[i]);
  }
  return s;
}

console.time("a");
sum([1, 2, 3, 4, 5, 6, 7, 8]).then((v) => {
  console.log(v); // 36
  console.timeEnd("a"); // a: <耗时>
});

export default {};
```

39. 请实现 mergeArray 方法

```
/**
```

```
 * @file 合并两个有序数组
 */

function merge(arr: number[], arr2: number[]): number[] {
  // 参考答案
  return arr.concat(arr2).sort((a: number, b: number) => a - b);
}

// 参数数组从小到大排列
console.log(merge([1, 2, 3], [2, 5, 6])); // [ 1, 2, 2, 3, 5, 6 ]

export default {};
```

40. 实现 firstSingleChar 方法

```
/**
 * @file 找出字符串中第一个只出现一次的字符
 */

function firstSingleChar(str: string) {
  // 参考答案
  return str.split("").filter((item: string, index: number, arr: string[]) =>
{
    arr.splice(index, 1);
    return !arr.includes(item);
  })[0];
}

// a 和 b 都出现了两次，只有 c 出现了一次，返回 c
console.log(firstSingleChar("abcba")); // c
// b c d 都出现了一次，返回第一个
console.log(firstSingleChar("aabcdee")); // b
// a 和 b 都出现了多次，没有只出现一次的元素，返回 undefined
console.log(firstSingleChar("aaaabbbb")); // undefined
console.log(firstSingleChar("dabvb"));

export default {};
```

41. 实现下面的 reverseWord 方法

```
/**
 * @file 反转句子
```

```
 *
 * 同时满足以下条件：1、去除首尾空格，2、单词间隔中多个空格变成一个；
 * 注意console示例运行结果
 */

function reverseWord(str: string):string {
    // 参考答案
    return (<string[]>str.match(/\S+/g)).reverse().join(" ");
}

console.log(reverseWord('the sky is blue')); // blue is sky the
// 去除首尾空格
console.log(reverseWord("  hello world  ")); // world hello
// 单词间隔中多个空格变成一个
console.log(reverseWord("a good   example")); // example good a

export default {}
```

42. 如何定义一个数组，它的元素可能是字符串类型，也可能是数值类型？

```
 // 方法1:
let arr1: (number | string)[] = [1]
arr1.push(1)
arr1.push('3')

// 方法2:
let  arr2 : Array<string | number> = [1, '2']
arr2.push(1)
arr2.push('3')

// 方法3:
type newType = number|string
let arr3:newType []= [3]
arr3.push(1)
arr4.push('5')
```

43. 请补充 objToArray 函数

```
/**
 * @file objToArray
 *
```

```
 * 将对象按照要求转为数组
 * 注意console示例运行结果
 */
type Obj = Record<string, string>;
interface FormatItem {
  key: string;
  op: string;
  value: string;
}

function objToArray(obj: Record<string, Obj>): FormatItem[] {
  return Object.keys(obj).reduce((value: Array<FormatItem>, key: string) => {
    var op: string = Object.keys(obj[key])[0];
    value.push({ key: key, op: op, value: obj[key][op] });
    return value;
  }, []);
}

console.log(
  objToArray({
    key1: {
      op1: "value1",
    },
    key2: {
      op2: "value2",
    },
  })
);
// result示例
// [
//     {key: 'key1', op: 'op1', value: 'value1'},
//     {key: 'key2', op: 'op2', value: 'value2'}
// ]

export default {};
```

44. 使用TS实现一个判断传入参数是否是数组类型的方法

unknown 用于变量类型不确定，但肯定可以确定的情形下，比如下面这个示例中，参数总归会有个值，根据这个值的类型进行不同的处理，这里使用 unknown 替代 any 则会更加类型安全。

```
function isArray(x: unknown): boolean {
        return Array.isArray(x)
}
```

45. TypeScript 的内置数据类型有哪些?

- boolean：表示布尔值，可以是 true 或 false。
- number：表示数字，包括整数和浮点数。
- string：表示字符串。可以使用单引号或双引号来表示字符串。
- void：表示没有任何返回值的函数的返回类型。
- null 和 undefined：这两个类型是所有类型的子类型。 symbol：表示独特的值，类似于数字或字符串。

除此之外，TypeScript 还支持以下复合类型：

- array：表示一个元素类型为 T 的数组。例如，number[] 表示一个数字数组。
- tuple：表示已知元素数量和类型的数组。例如，[string, number] 表示一个字符串和数字组成的元组。
- enum：表示一个命名的常量枚举。
- any：表示任意类型。
- unknown：与 any 类似，但是在更严格的类型检查下使用。
- object：表示非原始类型的对象。
- 还有一些其他的类型，例如 never、union 和 intersection，它们可以用于描述更复杂的类型。

46. ts中any和unknown有什么区别?

unknown 和 any 的主要区别是 unknown 类型会更加严格：在对 unknown 类型的值执行大多数操作之前，我们必须进行某种形式的检查。而在对 any 类型的值执行操作之前，我们不必进行任何检查。

举例说明：

```
let foo: any = 123;
console.log(foo.msg); // 符合TS的语法
let a_value1: unknown = foo;   // OK
let a_value2: any = foo;      // OK
let a_value3: string = foo;   // OK

let bar: unknown = 222; // OK
console.log(bar.msg); // Error
let k_value1: unknown = bar;   // OK
let K_value2: any = bar;      // OK
let K_value3: string = bar;   // Error
```

因为bar是一个未知类型(任何类型的数据都可以赋给 `unknown` 类型)，所以不能确定是否有msg属性。不能通过TS语法检测；而 unknown 类型的值也不能将值赋给 any 和 unknown 之外的类型变量

总结

any 和 unknown 都是顶级类型，但是 unknown 更加严格，不像 any 那样不做类型检查，反而 unknown 因为未知性质，不允许访问属性，不允许赋值给其他有明确类型的变量。

47. 如何将 unknown 类型指定为一个更具体的类型?

- 使用 typeof 进行类型判断（这些缩小类型范围的技术都有助于TS基于控制流程下的类型分析）

```
function unknownToString(value: unknown): string {
  if (typeof value === "string") {
   return value;
  }

  return String(value);
}
```

- 对 unknown 类型使用类型断言

要强制编译器信任类型为 unknown 的值为给定类型，则可以使用类型断言：

```
const value: unknown = "Hello World";
const foo: string = value; // Error
const bar: string = value as string; // OK
```

断言错了时语法能通过检测，但是运行的时候就会报错了！

```
const value: unknown = "Hello World";

const bar: number = value as number; // runtime Error
```

48. 使用ts实现一个判断入参是否是数组类型的方法?

unknown 用于变量类型不确定，但肯定可以确定的情形下，比如下面这个示例中，入参总归会有个值，根据这个值的类型进行不同的处理，这里使用 unknown 替代 any 则会更加类型安全。

```
function isArray(x: unknown): boolean {
  if (Array.isArray(x)) {
    return true;
  }
  return false;
```

```
}
```

49. tsconfig.json文件有什么用?

```
tsconfig.json文件是JSON格式的文件。

在tsconfig.json文件中，可以指定不同的选项来告诉编译器如何编译当前项目。

目录中包含tsconfig.json文件，表明该目录是TypeScript项目的根目录。

// 常用配置
{
  /*
      tsconfig.json是ts编译器的配置文件，ts可以根据它的信息来对待吗进行编译 可以再tsconfig中写注释
      include : 用来指定哪些文件需要被编译
      exclude : 用来指定哪些文件不需要被编译 : 默认node_module
      extends : 用来指定继承的配置文件
      files   : 用来指定被编译的文件列表，只有编译少量文件才使用
      compilerOptions : 编译器的选项是配置文件中非常重要也是非常复杂的配置选项
  */
  "include":[
    // ** : 任意目录 ,  * : 任意文件
    "./src/**/*"
  ],
  "exclude": [
    "./src/hello/**/*"
  ],
  // "extends": "./configs/base",
  "files": [
    "1.ts",
    // "2.ts"
  ],
  "compilerOptions": {
    // 用来指定 ES 版本 ESNext : 最新版。 'ES3', 'ES5', 'ES6'/'ES2015', 'ES2016', 'ES2017', 'ES2018', 'ES2019', 'ES2020', 'ESNext'
    "target": "ES2020",
    // 指定要使用模块化的规范 : 'None', 'CommonJS', 'AMD', 'System', 'UMD', 'ES6'/'ES2015', 'ES2020' or 'ESNext'
    "module": "ESNext",
    // 用来指定项目中要使用的库 'ES5', 'ES6', 'ES2015', 'ES7', 'ES2016', 'ES2017', 'ES2018', 'ESNext', 'DOM', 'DOM.Iterable',
    //                          'WebWorker', 'ScriptHost', 'ES2015.Core', 'ES2015.Collection', 'ES2015.Generator', 'ES2015.Iterable',
```

```
    //                          'ES2015.Promise', 'ES2015.Proxy', 'ES2015.Reflect', 'ES2015.Symbol', 'ES2015.Symbol.WellKnown',
    //                          'ES2016.Array.Include', 'ES2017.object', 'ES2017.Intl', 'ES2017.SharedMemory', 'ES2017.String',
    //                          'ES2017.TypedArrays', 'ES2018.Intl', 'ES2018.Promise', 'ES2018.RegExp', 'ESNext.AsyncIterable',
    //                          'ESNext.Array', 'ESNext.Intl', 'ESNext.Symbol'
    // 运行在浏览器中不用设置，运行在node或其他中才需要设置
    // "lib":[],
    // 用来指定编译后文件的存放位置
    "outDir":"./dist",
    // 将代码合并为一个文件,设置之后所有的全局作用域中的代码会合并到同一个文件中 但是只能在  'amd' and 'system' 中才能使用
    // "outFile": "./dist/app.js",
    // 是否对js文件进行编译，默认false
    "allowJs": false,
    // 是否检查js代码是否符合语法规范，默认false
    "checkJs": false,
    // 是否移除注释，默认false
    "removeComments":false,
    // 是否不生成编译后文件，默认false
    "noEmit": false,
    // 当有错误时是否生成文件，默认false
    "noEmitOnError": false,
    // 是否生成sourceMap，默认false  这个文件里保存的，是转换后代码的位置，和对应的转换前的位置。有了它，出错的时候，通过断点工具可以直接显示原始代码，而不是转换后的代码。
    "sourceMap":false,

    // 所有的严格检查的总开关，默认false
    "strict": false,
    // 编译后的文件是否开启严格模式，默认false
    "alwaysStrict": false,
    // 不允许隐式的any，默认false(允许)
    "noImplicitAny": false,
    // 不允许隐式的this，默认false(允许)
    "noImplicitThis": false,
    // 是否严格的检查空值，默认false 检查有可能为null的地方
    "strictNullChecks": true,
    // 是否严格检查bind、call和apply的参数列表，默认false  检查是否有多余参数
    "strictBindCallApply":false,
    // 是否严格检查函数的类型，
    "strictFunctionTypes":false,
    // 是否严格检查属性是否初始化，默认false
    "strictPropertyInitialization":false,

    // 是否检查switch语句包含正确的break，默认false
    "noFallthroughCasesInSwitch":false,
```

```
    // 检查函数没有隐式的返回值，默认false
    "noImplicitReturns":false,
    // 是否检查检查未使用的局部变量，默认false
    "noUnusedLocals":false,
    // 是否检查未使用的参数，默认false
    "noUnusedParameters":false,

    // 是否检查不可达代码报错，默认false   true，忽略不可达代码 false，不可达代码将引起错误
    "allowUnreachableCode":false
  }
}
```

50. TypeScript中的 Declare 关键字有什么用?

JavaScript库或框架没有TypeScript声明文件。 但是，如果要在TypeScript文件中使用它们而没有任何编译错误，则必须使用declare关键字。 declare关键字用于环境声明和您要定义可能在其他位置存在的变量的方法。

如果要在我们的TypeScript代码中使用该库，则可以使用以下代码：

```
declare var myLibrary;
```

TypeScript运行时会将myLibrary变量分配为 any。

51. 解释一下TypeScript中的枚举。

枚举是TypeScipt数据类型，它允许我们定义一组命名常量。 使用枚举使记录意图或创建一组不同的案例变得更加容易。 它是相关值的集合，可以是数字值或字符串值。

```
enum Gender {
    Male,
    Female
    Other
}
console.log(Gender.Male); // Output: 0

//We can also access an enum value by it's number value.
console.log(Gender[1]); // Output: Female
```

52. TypeScript 的主要特点是什么?

- 跨平台：TypeScript 编译器可以安装在任何操作系统上，包括 Windows、macOS 和 Linux。
- ES6 特性：TypeScript 包含计划中的 ECMAScript 2015 (ES6) 的大部分特性，例如箭头函数。
- 面向对象的语言：TypeScript 提供所有标准的 OOP 功能，如类、接口和模块。
- 静态类型检查：TypeScript 使用静态类型并帮助在编译时进行类型检查。因此，你可以在编写代码时发现编译时错误，而无需运行脚本。
- 可选的静态类型：如果你习惯了 JavaScript 的动态类型，TypeScript 还允许可选的静态类型。
- DOM 操作：您可以使用 TypeScript 来操作 DOM 以添加或删除客户端网页元素。

53. TypeScript中的方法重写是什么?

如果子类(子类)具有与父类中声明的相同的方法，则称为方法覆盖。换句话说，在派生类或子类中重新定义基类方法。

方法重写的规则

- 该方法必须具有与父类相同的名称
- 该方法必须具有与父类相同的参数。
- 必须有一个IS-A关系(继承)。

例子

```
class NewPrinter extends Printer {

    doPrint(): any {

        super.doPrint();

        console.log("Called Child class.");

    }

    doInkJetPrint(): any {

        console.log("Called doInkJetPrint().");

    }
}
let printer: new () => NewPrinter;
printer.doPrint();
printer.doInkJetPrint();
```

54. 什么是TypeScript映射文件?

- TypeScript Map文件是一个源映射文件，其中包含有关我们原始文件的信息。
- .map文件是源映射文件，可让工具在发出的JavaScript代码和创建它的TypeScript源文件之间进行映射。
- 许多调试器可以使用这些文件，因此我们可以调试TypeScript文件而不是JavaScript文件。

55. TypeScript中的类型有哪些?

参考答案:

类型系统表示语言支持的不同类型的值。它在程序存储或操作所提供的值之前检查其有效性。

它可以分为两种类型，

- 内置：包括数字(number)，字符串(string)，布尔值(boolean)，无效(void)，空值(null)和未定义(undefined)。
- 用户定义的：它包括枚举(enums)，类(classes)，接口(interfaces)，数组(arrays)和元组(tuple)。

56. 如何检查TypeScript中的null和undefined？

通过使用一个缓冲检查，我们可以检查空和未定义:

```
if (x == null) {
}
```

如果我们使用严格的检查，它将总是对设置为null的值为真，而对未定义的变量不为真。

例子

```
var a: number;
var b: number = null;
function check(x, name) {

    if (x == null) {

        console.log(name + ' == null');

    }

    if (x === null) {

        console.log(name + ' === null');

    }

```

```
    if (typeof x === 'undefined') {

        console.log(name + ' is undefined');

    }
}
check(a, 'a');
check(b, 'b');
```

输出

```
"a == null"
"a is undefined"
"b == null"
"b === null"
```

57. 如何在TypeScript中实现继承?

继承是一种从另一个类获取一个类的属性和行为的机制。它是OOPs语言的一个重要方面，并且具有从现有类创建新类的能力，继承成员的类称为基类，继承这些成员的类称为派生类。

继承可以通过使用extend关键字来实现。我们可以通过下面的例子来理解它。

```
class Shape {

   Area:number

   constructor(area:number) {

      this.Area = area

   }
}
class Circle extends Shape {

   display():void {

      console.log("圆的面积: "+this.Area)

   }
}
var obj = new Circle(320);
obj.display()
```

58. 什么是TypeScript Declare关键字?

我们知道所有的JavaScript库/框架都没有TypeScript声明文件，但是我们希望在TypeScript文件中使用它们时不会出现编译错误。为此，我们使用declare关键字。在我们希望定义可能存在于其他地方的变量的环境声明和方法中，可以使用declare关键字。

例如，假设我们有一个名为myLibrary的库，它没有TypeScript声明文件，在全局命名空间中有一个名为myLibrary的命名空间。如果我们想在TypeScript代码中使用这个库，我们可以使用以下代码:

```
declare var myLibrary;
```

TypeScript运行时将把myLibrary变量赋值为任意类型。这是一个问题，我们不会得到智能感知在设计时，但我们将能够使用库在我们的代码。

59. TypeScript 和 JavaScript 的区别是什么?

Typescript 是 JavaScript 的超集，可以被编译成 JavaScript 代码。用 JavaScript 编写的代码，在 TypeScript 中依然有效。Typescript 是纯面向对象的编程语言，包含类和接口的概念。 程序员可以用它来编写面向对象的服务端或客户端程序，并将它们编译成 JavaScript 代码。

60. Typescript中什么是类类型接口?

- 如果接口用于一个类的话，那么接口会表示“行为的抽象”
- 对类的约束，让类去实现接口，类可以实现多个接口
- 接口只能约束类的公有成员（实例属性/方法），无法约束私有成员、构造函数、静态属性/方法

61. Typescript中never 和 void 的区别?

- void 表示没有任何类型（可以被赋值为 null 和 undefined）。
- never 表示一个不包含值的类型，即表示永远不存在的值。
- 拥有 void 返回值类型的函数能正常运行。拥有 never 返回值类型的函数无法正常返回，无法终止，或会抛出异常。

62. Typescript中 interface 和 type 的差别是什么?

相同点

- 都可以描述一个对象或者函数

interface

```
interface User {
  name: string
  age: number
}

interface SetUser {
  (name: string, age: number): void;
}
```

type

```
type User = {
  name: string
  age: number
};

type SetUser = (name: string, age: number)=> void;
```

- 都允许拓展（extends）

interface 和 type 都可以拓展，并且两者并不是相互独立的，也就是说 interface 可以 extends type, type 也可以 extends interface 。 虽然效果差不多，但是两者语法不同。

interface extends interface

```
interface Name {
  name: string;
}
interface User extends Name {
  age: number;
}
```

type extends type

```
type Name = {
  name: string;
}
type User = Name & { age: number  };
```

interface extends type

```
type Name = {
  name: string;
}
interface User extends Name {
  age: number;
}
```

type extends interface

```
interface Name {
  name: string;
}
type User = Name & {
  age: number;
}
```

不同点

- type 可以而 interface 不行

type 可以声明基本类型别名，联合类型，元组等类型

```
// 基本类型别名
type Name = string

// 联合类型
interface Dog {
    wong();
}
interface Cat {
    miao();
}

type Pet = Dog | Cat

// 具体定义数组每个位置的类型
type PetList = [Dog, Pet]
```

type 语句中还可以使用 typeof 获取实例的 类型进行赋值

```
// 当你想获取一个变量的类型时，使用 typeof
let div = document.createElement('div');
```

```
type B = typeof div
```

其他骚操作

```
type StringOrNumber = string | number;
type Text = string | { text: string };
type NameLookup = Dictionary<string, Person>;
type Callback<T> = (data: T) => void;
type Pair<T> = [T, T];
type Coordinates = Pair<number>;
type Tree<T> = T | { left: Tree<T>, right: Tree<T> };
```

- interface 可以而 type 不行

interface 能够声明合并

```
interface User {
  name: string
  age: number
}

interface User {
  sex: string
}

/*
User 接口为 {
  name: string
  age: number
  sex: string
}
*/
```

一般来说，如果不清楚什么时候用interface/type，能用 interface 实现，就用 interface , 如果不能就用 type 。

63. 说一说TypeScript中的类及其特性。

TypeScript 引入了类，以便它们可以利用诸如封装和抽象之类的面向对象技术的好处。

TypeScript 编译器将 TypeScript 中的类编译为普通的 JavaScript 函数，以跨平台和浏览器工作。

一个类包括以下内容：

- 构造器（Constructor）
- 属性（Properties）
- 方法（Methods）

```
class Employee {
    empID: number;
    empName: string;
    constructor(ID: number, name: string) {
        this.empName = name;
        this.empID = ID;
    }
    getSalary(): number {
        return 40000;
    }
}
```

类的其他特性有：

- 继承（Inheritance）
- 封装（Encapsulation）
- 多态（Polymorphism）
- 抽象（Abstraction）

64. TypeScript中的变量以及如何声明?

变量是内存中用于存储值的命名空间。

在 TypeScript 中声明变量的类型语法在变量名称后包括一个冒号（:)，后跟其类型。 与 JavaScript 相似，我们使用var关键字声明变量。

在Typescript中声明变量时，必须遵循某些规则：

- 变量名称必须是字母或数字。
- 不能以数字开头名称。
- 除下划线（ _ ）和美元（ $ ）符号外，它不能包含空格和特殊字符。

65. TypeScript 中的类是什么？你如何定义它们?

类表示一组相关对象的共享行为和属性。

例如，我们的类可能是Student，其所有对象都具有该attendClass方法。另一方面，John是一个单独的 type 实例，Student可能有额外的独特行为，比如attendExtracurricular.

你使用关键字声明类class：

```
class Student {

    studCode: number;

    studName: string;

    constructor(code: number, name: string) {

            this.studName = name;

            this.studCode = code;
    }
 }
```

66. TypeScript 中的 getter/setter 是什么？你如何使用它们？

Getter 和 setter 是特殊类型的方法，可帮助你根据程序的需要委派对私有变量的不同级别的访问。

Getters 允许你引用一个值但不能编辑它。Setter 允许你更改变量的值，但不能查看其当前值。这些对于实现封装是必不可少的。

例如，新雇主可能能够了解get公司的员工人数，但无权set了解员工人数。

```
const fullNameMaxLength = 10;
class Employee {
  private _fullName: string = "";
  get fullName(): string {
    return this._fullName;
  }
  set fullName(newName: string) {
    if (newName && newName.length > fullNameMaxLength) {
      throw new Error("fullName has a max length of " + fullNameMaxLength);
    }
    this._fullName = newName;
  }
}
let employee = new Employee();
employee.fullName = "Bob Smith";
if (employee.fullName) {
  console.log(employee.fullName);
}
```

67. Typescript中什么是装饰器，它们可以应用于什么？

装饰器是一种特殊的声明，它允许你通过使用@<name>注释标记来一次性修改类或类成员。每个装饰器都必须引用一个将在运行时评估的函数。

例如，装饰器@sealed将对应于sealed函数。任何标有 的@sealed都将用于评估sealed函数。

```
function sealed(target) {
  // do something with 'target' ...
}
```

它们可以附加到：

- 类声明
- 方法
- 配件
- 特性
- 参数

注意：默认情况下不启用装饰器。要启用它们，你必须 `experimentalDecorators`从 `tsconfig.json` 文件或命令行编辑编译器选项中的字段。

68. 解释如何使用 TypeScript mixin。

Mixin 本质上是在相反方向上工作的继承。Mixins 允许你通过组合以前类中更简单的部分类设置来构建新类。

相反，类A继承类B来获得它的功能，类B从类A需要返回一个新类的附加功能。

69. TypeScript 中的类型断言是什么?

TypeScript 中的类型断言的工作方式类似于其他语言中的类型转换，但没有 C# 和 Java 等语言中可能的类型检查或数据重组。类型断言对运行时没有影响，仅由编译器使用。

类型断言本质上是类型转换的软版本，它建议编译器将变量视为某种类型，但如果它处于不同的形式，则不会强制它进入该模型。

70. TypeScript 中的模块是什么?

TypeScript 中的模块是相关变量、函数、类和接口的集合。 你可以将模块视为包含执行任务所需的一切的容器。可以导入模块以轻松地在项目之间共享代码。

```
module module_name{
  class xyz{
    export sum(x, y){
```

```
        return x+y;
      }
    }
  }
```

71. 为什么推荐使用 TypeScript？

TypeScript是微软公司开发和维护的一种面向对象的编程语言。它是JavaScript的超集，包含其所有元素。

强类型和弱类型、静态类型和动态类型是两组不同的概念。

类型强弱是针对类型转换是否显示来区分，静态和动态类型是针对类型检查的时机来区分。

TS对JS的改进主要是静态类型检查，静态类型检查有何意义？标准答案是“静态类型更有利于构建大型应用”。

推荐使用TypeScript的原因有：

- TypeScript简化了JavaScript代码，使其更易于阅读和调试。
- TypeScript是开源的。
- TypeScript为JavaScript ide和实践（如静态检查）提供了高效的开发工具。
- TypeScript使代码更易于阅读和理解。
- 使用TypeScript，我们可以大大改进普通的JavaScript。
- TypeScript为我们提供了ES6（ECMAScript 6）的所有优点，以及更高的生产率。
- TypeScript通过对代码进行类型检查，可以帮助我们避免在编写JavaScript时经常遇到的令人痛苦的错误。
- 强大的类型系统，包括泛型。
- TypeScript只不过是带有一些附加功能的JavaScript。
- TypeScript代码可以按照ES5和ES6标准编译，以支持最新的浏览器。
- 与ECMAScript对齐以实现兼容性。
- 以JavaScript开始和结束。
- 支持静态类型。
- TypeScript将节省开发人员的时间。
- TypeScript是ES3、ES5和ES6的超集。